

# Análisis de datos estructurados mediante



# Link al colab de Pandas

# Qué es Pandas

- Es una librería orientada al trabajo con DataFrames (estructuras tabulares) desarrollada inicialmente en 2008 por Wes McKinney.
- Es de código abierto.
- Es multiplataforma.
- Se encuentra optimizado con altos estándares de calidad.
- Sintaxis de alto nivel.

# Funciones más utilizadas de Pandas

`read_csv()`: permite abrir csv.

`read_excel()`: permite abrir un excel.

`read_json()`: permite cargar un json.

`DataFrame()`: crea un dataframe.

`to_datetime()`: convierte a fecha.

`merge()`: une dos tablas.

# Estructuras de datos básicas en Pandas

## Series:

Vectores unidimensionales etiquetados que contienen datos de cualquier tipo. Está basada en ndarray.

## DataFrame:

Estructura de datos bi-dimensional que puede contener tanto arrays de dos dimensiones o tablas con filas y columnas.

# Atributos de las Series

index: índices del objeto.

values: valores del objeto.

name: es un atributo opcional con el nombre de la serie.

is\_unique: retorna True o False según si los valores son únicos o no.

posee la mayoría de los atributos de ndarray (no data, itemsize y strides)

# Métodos de las series

Posee la mayoría de los métodos de ndarray (flatten y reshape no) y agrega funciones para la exploración, manipulación y transformación de datos.

head(): muestra los primeros n elementos.

describe(): retorna medidas de tendencia central y otras estadísticas descriptivas.

dropna(): elimina valores nulos.

apply(): aplica una función a cada elemento.

Y muchos más...

# Atributos de los DataFrames

shape: devuelve cantidad de filas y columnas.

columns: devuelve nombre de las columnas.

dtypes: tipos de datos de cada columna.



# Métodos de los DataFrames

`to_csv()`: guarda un dataframe como csv.

`to_excel()`: guarda un dataframe como excel.

`head()`: muestra las primeras filas.

`info()`: retorna un resumen de qué contiene el dataframe.

`groupby()`: agrupa según una columna que recibe por parámetro.

# Selección de columnas

## Por nombre:

`df.id` o `df["id"]`

selecciona la columna de nombre "id" del dataframe llamado "df"

`df[["id", "damage"]]`

selecciona las columnas de nombre "id" y "damage" del dataframe llamado "df"

## Por posición:

`df.iloc[:, 1]`

Selecciona la primera columna del dataframe llamado "df"

`df.iloc[:, 1:4]`

Selecciona las primeras tres columnas del dataframe llamado df

# Filtrado de filas

## Por posición:

```
df.iloc[2, :]
```

retornará todas las columnas de la fila 2 del dataframe llamado “df”.

## Según condiciones:

```
df.loc[df.damage == 3, ["id", "damage"]]
```

retorna las columnas “id” y “damage” de todas las filas del df que en la columna damage tienen como valor 3.

```
df.query("damage > 2")
```

Retorna las filas de personajes con daño mayor a 2

# ¿Cómo funcionan los índices?

Los índices son mecanismos para simplificar el acceso a los datos. Como vimos previamente, los dataframes tienen un atributo `index` que podemos setear o acceder. Tanto `loc` como `iloc` nos permiten el acceso a distintas posiciones del dataframe. `loc` utiliza los nombres de las filas y las columnas, mientras que `iloc` utiliza posiciones numéricas.

# Procesamiento de fechas

Con la función `to_datetime()` se puede transformar una columna en formato fecha y luego mediante `dt` acceder al año, el mes o el día.

# Procesamiento de cadenas de texto

Pandas también simplifica el procesamiento de cadenas de texto. Mediante el accesor *str* se puede trabajar con cadenas de texto en vectores y aplicar funciones como *lower()*, *splice()*, *upper()*, *split()*, *replace()*, etc.